Academy Linker
API Interface Specification

Draft v1.0

Document status

This document consolidates the final interface decisions discussed for the first implementation phase. It is a product-facing API specification draft, not a full database schema or OpenAPI export. Unsettled fields are marked as TBD, but all global conventions and security rules below are intended to be binding for implementation.

Item	Decision
Service base path	/api
Primary consumer	React web client, with Postman and similar API tools supported for testing
Naming style	All JSON fields use snake_case; resource names in paths use plural nouns
Authentication	JWT with access token and refresh token stored in HttpOnly cookies
Access token lifetime	15 minutes
Refresh token lifetime	3 days
Refresh rotation	Not used in phase 1; refresh token remains valid until expiry or explicit revocation
Time storage	All backend and database timestamps are stored in UTC
Internationalization	Content objects store original text, translated text, language metadata, translation_status, and translated_at

1. Scope and design goals

- Provide a clear interface contract for parent, teacher, and admin clients.
- Keep page-level frontend logic thin by exposing aggregate endpoints for dashboard-style pages.

- Use resource-oriented APIs as the default style, with explicit action endpoints only where state transition semantics are clearer than generic PATCH operations.
- Keep the model secure even though external identifiers are routable and human-visible student sid values may be repeated across schools.

2. Final domain constraints

Item	Decision
User role model	One user may hold exactly one role in phase 1. Mixed parent plus teacher accounts are not allowed.
Student identity	Each student record has an internal auto-increment database id, a public uuid used by APIs and routes, and a school sid used only for display and search.
Parent-student relationship	Current product logic is effectively one parent to one student, but the database must retain a binding table so the model can later expand to one-to-many or many-to-many.
Teacher-student relationship	Teachers may access only directly assigned students.
Discussion container	Exactly one fixed discussion container exists per parent-teacher pair in phase 1. Users cannot create additional threads.
Tags	Tags apply only to posts. System tags such as important are shared school-wide; other tags created by a teacher are private to that teacher. Parents cannot create tags.
Per-user state	Read and archive status are personal user actions, not global resource state. User-state join tables are required.

3. Global API conventions

3.1 Resource naming and path layout

- All endpoints are rooted under /api.
- Role-specific APIs use explicit role paths where that improves permission clarity: /api/parents/me/..., /api/teachers/me/..., and /api/admin/... .
- Shared account APIs use /api/me and /api/me/settings.
- Path parameters always use public uuid values, never internal database ids and never sid values.

3.2 Request and response envelopes

Successful responses always wrap the main payload in a top-level data field. List responses also include a top-level meta object for paging and related metadata.

Successful object response example

```
{
  "data": {
    "student_uuid": "4b6dcb68-8a89-4fd4-9580-15e41254cb8b",
    "sid": "A-10392",
    "name": "Alex Chen"
  }
}
```

Successful list response example

```
{
  "data": [
    {"report_uuid": "..."},
    {"report_uuid": "..."}
  ],
  "meta": {
    "page": 1,
    "page_size": 20,
    "total": 52
  }
}
```

Error response example

```
{
  "error": {
    "code": "FORBIDDEN",
    "message": "You do not have access to this student.",
    "details": null
  }
}
```

3.3 Pagination, filtering, sorting, and search

- Server-side filtering and sorting are the default. The frontend passes filter and sort conditions; the backend returns only the requested page of results.
- Baseline query parameters: page, page_size, sort, and keyword.
- Additional filters may be added per resource, such as tag, class_name, grade, status, date_from, and date_to.
- Sort values should be explicit string enums such as created_at_desc, created_at_asc, score_desc, or name_asc.
- The backend is responsible for enforcing permissions before filtering or sorting results.

3.4 Time, language, and translation fields

- All persisted timestamps are stored in UTC and returned as ISO 8601 strings.
- Display timezone is determined from user settings when configured; otherwise the client may fall back to browser timezone for presentation only.
- Language selection priority is: explicit user setting, then browser language, then system default.
- Content resources that support translation should provide original text, translated text, original_language, translated_language, translation_status, and translated_at where applicable.

4. Authentication and security

4.1 Selected authentication model

- Authentication uses JWT access tokens and refresh tokens stored in cookies.
- Access token lifetime is 15 minutes.
- Refresh token lifetime is 3 days.
- Refresh token rotation is not used in phase 1. When the refresh token expires, the user must log in again.
- Normal business requests are authenticated with the access token only. The refresh token is sent only to the refresh endpoint.

4.2 Cookie model and attribute choices

Item	Decision
Cookie concept	A cookie is a browser-stored key-value value that the browser automatically attaches to matching requests. In this project cookies carry the access token and refresh token so frontend JavaScript does not need to read or store them directly.
Access token cookie path	Path=/ . The browser may attach the access token cookie to application requests across the site.
Refresh token cookie path	Path=/api/auth/refresh . This narrows refresh token exposure so it is sent only when requesting a new access token.
HttpOnly	Enabled. Frontend JavaScript must not and cannot read these cookies directly.
Secure	Enabled in environments served over HTTPS.
SameSite	Lax. The browser will not send these cookies on most cross-site background requests, which reduces CSRF risk while preserving common navigation flows.

4.3 SameSite and Origin checks

SameSite controls whether the browser sends cookies when the request originates from another site. Lax is selected as the phase 1 balance between compatibility and CSRF risk reduction.

Origin is a browser-supplied request header containing the initiating scheme, host, and port. For state-changing endpoints the backend should validate Origin against an allowlist of trusted frontend origins before processing the request.

Frontend security notes

Clients must send requests with credentials enabled. In browser code this typically means fetch(..., { credentials: "include" }). Frontend code must never attempt to read token cookies, persist token values in localStorage, or re-implement authentication state from raw token contents.

4.4 Logout behavior and residual access window

- The service supports logout of the current device and logout of all devices.
- Logout revokes refresh tokens immediately.
- Because access tokens are stateless JWTs and phase 1 does not include token_version or revoked_after checks, an already issued access token may remain usable until its 15-minute expiry.
- This residual window is accepted in phase 1 in exchange for simpler implementation.

5. Authorization rules

Role	Can access	Cannot access	Notes
Parent	Only bound student records, related reports, announcements, dashboard data, and discussions with assigned teachers	Other students, teacher-only data, other parents' settings, teacher-owned tag administration	Parents may create posts but may not delete teacher posts.
Teacher	Only directly assigned students, their relevant parents for discussion, posts within authorized discussion containers, and teacher-private tags	Unassigned students, parent account settings, other teachers' private tags	Teachers may apply business tags such as important when permitted by tag policy.
Admin	Administrative management endpoints defined under /api/admin	Role-specific restrictions do not apply, subject to admin policy	Admin scope should remain explicit and auditable.

6. Resource model notes

- student_uuid is the external identity key for route paths and cross-resource references. sid is display-oriented and may be non-unique across source schools.
- DiscussionThread in phase 1 acts as a fixed conversation container for one parent-teacher pair rather than a user-created topic thread.
- Posts are listed in a flat ordered stream within the discussion container, subject to pagination, sorting, filtering, and search.
- Tag logic belongs to posts, not to the discussion container itself.
- important is a system tag that may drive dashboard banners and other business behavior; it should be implemented as structured tag metadata, not only a hard-coded string check.
- User-specific read and archive state must be stored in join tables such as user_report_states and user_announcement_states.

7. API endpoints

7.1 Authentication and account endpoints

Method	Path	Purpose	Auth
POST	/api/auth/login	Authenticate credentials and issue token cookies	Public
POST	/api/auth/refresh	Issue a new access token using the refresh token cookie	Refresh cookie
POST	/api/auth/logout	Log out the current device by revoking its refresh token and clearing cookies	Access cookie
POST	/api/auth/logout_all	Revoke all refresh tokens for the current user and clear current cookies	Access cookie
GET	/api/me	Return current account profile and role summary	Access cookie
GET	/api/me/settings	Return current user settings	Access cookie
PATCH	/api/me/settings	Update current user settings	Access cookie

7.2 Parent endpoints

Method	Path	Purpose	Auth
GET	/api/parents/me/students	List bound students visible to the current parent	Parent
GET	/api/parents/me/students/{student_uuid}/dashboard	Aggregate dashboard data for one student	Parent
GET	/api/parents/me/students/{student_uuid}/subjects	List student subjects visible to the parent	Parent
GET	/api/parents/me/students/{student_uuid}/subjects/{subject_uuid}	Return one student-subject detail view	Parent
GET	/api/parents/me/students/{student_uuid}/reports	List reports for one student with paging and filters	Parent
GET	/api/parents/me/students/{student_uuid}/reports/{report_uuid}	Return a single report detail object	Parent

Method	Path	Purpose	Auth
POST	/api/parents/me/students/{student_uuid}/reports/{report_uuid}/read	Mark report as read for current parent only	Parent
POST	/api/parents/me/students/{student_uuid}/reports/{report_uuid}/archive	Archive report for current parent only	Parent
POST	/api/parents/me/students/{student_uuid}/reports/{report_uuid}/unarchive	Unarchive report for current parent only	Parent
GET	/api/parents/me/students/{student_uuid}/announcements	List announcements and tasks visible to the parent	Parent
GET	/api/parents/me/students/{student_uuid}/teachers	List teachers relevant to the student for discussion entry points	Parent
GET	/api/parents/me/students/{student_uuid}/discussions/teachers/{teacher_uuid}	Return the fixed parent-teacher discussion container and paged post stream	Parent
POST	/api/parents/me/students/{student_uuid}/discussions/teachers/{teacher_uuid}/posts	Create a new parent-authored post	Parent

7.3 Teacher endpoints

Method	Path	Purpose	Auth
GET	/api/teachers/me/students	List students directly assigned to the teacher with filters, sort, and search	Teacher
GET	/api/teachers/me/students/{student_uuid}/dashboard	Return teacher-facing aggregate student overview data	Teacher
GET	/api/teachers/me/students/{student_uuid}/parents	List parents linked to the student for discussion access	Teacher
GET	/api/teachers/me/students/{student_uuid}/discussions/parents/{parent_uuid}	Return the fixed teacher-parent discussion container and paged post stream	Teacher

Method	Path	Purpose	Auth
POST	/api/teachers/me/students/{student_uuid}/discussions/parents/{parent_uuid}/posts	Create a new teacher-authored post	Teacher

Teacher post and tag endpoints

Method	Path	Purpose	Auth
PATCH	/api/teachers/me/posts/{post_uuid}	Update a teacher-authored post when allowed by policy	Teacher
DELETE	/api/teachers/me/posts/{post_uuid}	Delete a teacher-authored post when allowed by policy	Teacher
GET	/api/teachers/me/tags	List system tags and the current teacher's private tags	Teacher
POST	/api/teachers/me/tags	Create a new teacher-private tag	Teacher
PATCH	/api/teachers/me/tags/{tag_uuid}	Update a teacher-private tag	Teacher
DELETE	/api/teachers/me/tags/{tag_uuid}	Delete a teacher-private tag	Teacher

7.4 Admin endpoints

Method	Path	Purpose	Auth
GET	/api/admin/users	List users with filters and paging	Admin
POST	/api/admin/users	Create a user account	Admin
PATCH	/api/admin/users/{user_uuid}	Update a user account or role-scoped profile data	Admin
GET	/api/admin/students	List student records	Admin
POST	/api/admin/students	Create a student record	Admin
PATCH	/api/admin/students/{student_uuid}	Update a student record	Admin

Admin binding and system-tag endpoints

Method	Path	Purpose	Auth
POST	/api/admin/parent_student_bindings	Create or update a parent-student binding	Admin
POST	/api/admin/teaching_assignments	Create or update a teacher-student assignment	Admin
GET	/api/admin/system_tags	List school-wide system tags	Admin
POST	/api/admin/system_tags	Create a school-wide system tag	Admin
PATCH	/api/admin/system_tags/{tag_uuid}	Update a school-wide system tag	Admin

7.5 Aggregate endpoint policy

Aggregate endpoints are allowed and encouraged for page-level views such as dashboards. They may combine data from students, reports, announcements, summaries, and chart payloads into a single response so the frontend can render one page with minimal orchestration logic.

Aggregate endpoint constraint

Aggregate endpoints should coexist with standard resource endpoints. They are for page composition, not a replacement for all resource APIs.

8. Action endpoint policy

- PATCH is used for ordinary attribute updates.
- Explicit action endpoints are used when state changes are clearer than generic PATCH semantics, such as /archive, /unarchive, or /read.
- The API may therefore mix REST-style resource updates and action endpoints where appropriate. This is intentional.

9. Standard query parameter policy

Parameter	Type	Meaning	Examples
page	integer	1-based page index	1, 2, 3
page_size	integer	Page size selected by client within server cap	20, 50
sort	string enum	Explicit sort rule	created_at_desc, name_asc

Parameter	Type	Meaning	Examples
keyword	string	Free-text fuzzy search	alex, A-10392
tag	string or uuid	Filter by post tag	important

Parameter	Type	Meaning	Examples
status	string enum	Resource status filter	active, archived, unread
class_name	string	Class filter for teacher student lists	Year5-A
grade	string or integer	Grade filter	5
date_from/date_to	ISO datetime or date	Time range filter	2026-01-01T00:00:00Z

10. Recommended response fields by major resource

Item	Decision
Student	student_uuid, sid, name, class_name, grade, avatar_url, recent_update_at
Report	report_uuid, title, summary, content_markdown, original_language, translated_language, translation_status, translated_at, source_type, created_at
Announcement	announcement_uuid, title, content_markdown, tags, priority, starts_at, ends_at, created_at
Discussion container	thread_uuid, student_uuid, parent_uuid, teacher_uuid, latest_post_at, unread_count, participant_summary
Post	post_uuid, thread_uuid, author_user_uuid, author_role, content_markdown, tags, created_at, updated_at, reply_to_post_uuid
Tag	tag_uuid, name, scope, owner_teacher_uuid, affects_business_logic, is_selectable_by_parent, is_selectable_by_teacher

11. Frontend integration notes

- All browser requests that need authentication must include credentials so cookies are sent.

- On a 401 caused by expired access token, the client should attempt POST /api/auth/refresh and then retry the original request once. If refresh fails, redirect to login.
- Frontend code should treat sid as a display and search value only; all route generation and data references must use uuid values.
- Because report archive and read states are per-user, the frontend must not assume another user sees the same state for the same report.
- The current discussion model exposes one fixed discussion container per parent-teacher pair. The UI should not expose thread creation controls in phase 1.

12. Open implementation notes

- If the product later requires immediate access-token invalidation for logout-all, add token_version or revoked_after checks at request time.
- If parent-student relationships expand beyond 1:1, the binding-table approach already preserves a migration path with minimal API path changes.
- If future multilingual requirements exceed original plus translated, replace paired fields with a generalized translations table or localized content collection.
- If the discussion model later supports multiple topics, the current fixed discussion container can be extended into true multi-thread discussion without changing post tagging rules.